BLOCKCHAIN PROGRAMMING

Final Project Assignment

Online Banking System

1. Overview

You will build a blockchain-based term deposit system - similar to a bank savings account, but running entirely on a smart contract. Users lock their tokens for a fixed period, earn interest, and can withdraw when the term ends.

The system has two main roles:

- **Depositor (user)** - opens a deposit, earns interest, withdraws, or renews.
- **Bank Admin** - sets up saving plans, manages the interest vault, and can pause the system.

You will write two core contracts plus a mock token:

- **SavingCore** - handles saving plans and deposit certificates (ERC721 NFTs).
- **VaultManager** - holds the liquidity pool that pays interest to users.
- **MockUSDC** - a simple ERC20 token for testing (6 decimals).

Important - minimum, not maximum: these three contracts are only the required base. You are free to add more contracts, interfaces, or libraries, and you may change the internal design, as long as the user flows in Section 3 still work. A different architecture is not wrong - an unexplained one is. Explain every extra design choice in your README. See Section 8.3 for bonus ideas.

1.1 Why Three Contracts?

Before you write any code, you must understand why the system is split into three contracts. In a real bank, the money of customers, the money of the bank, and the business rules are kept separate. We follow the same idea:

Contract	What it does	Why it is a separate contract
MockUSDC.sol	A fake USDC token, 6 decimals, anyone can mint it for testing.	Real USDC cannot be minted by you. You need a test token with the same 6 decimals, so you find decimal bugs early (many students lose points because they assume 18 decimals).
VaultManager.sol	Holds the bank's own money (the interest pool). Admin can fund it, withdraw from it, set the fee receiver, and pause the system.	The bank's money must be separate from users' principal. If the interest pool is empty or has a bug, user deposits are still safe. It also gives one clear place for all admin money controls.
SavingCore.sol	All business logic: plans, open deposit, withdraw, renew (manual + auto), and the ERC721 certificate NFT.	Business rules change more often than money storage. Keeping logic in one contract and money in another makes the code easier to test, easier to audit, and closer to real-world design (separation of duties).

Key idea: SavingCore holds the users' principal. VaultManager holds the bank's interest money. Interest always comes from the vault, never from other users' principal. If you mix these two pools, your design is wrong.

2. Key Concepts

2.1 Saving Plan

A saving plan is a product the admin creates. Think of it as a bank offering: "deposit for 90 days and earn 2.5% APR." Each plan has:

Field	Meaning
tenorDays	How long the deposit lasts (e.g. 7, 30, 90, 180, 365 days)
aprBps	Annual interest rate in basis points (800 = 8.00% per year)
minDeposit / maxDeposit	Optional min/max amount. Zero means no limit.
earlyWithdrawPenaltyBps	Penalty if user withdraws before maturity (500 = 5%)
enabled	Admin can disable a plan to stop new deposits.

Hint: 1 basis point (bps) = 0.01%. So 250 bps = 2.50%.

2.2 Deposit Certificate (NFT)

When a user opens a deposit, the contract mints an ERC721 NFT representing their certificate. Each deposit records:

- The plan chosen, the amount deposited (principal), and the start/maturity timestamps.
- A snapshot of the APR and penalty rate at the time of opening - these are locked in and never change, even if the admin later updates the plan.
- A status: Active, Withdrawn, ManualRenewed, or AutoRenewed.

Why NFT? It makes the certificate transferable and uniquely identifiable on-chain.

3. User Flows

Below are the five main flows your contract must support. Read each one carefully - the math and the edge cases are the heart of this project.

3.1 Open a Deposit

User selects a plan and deposits an amount of tokens.

1. User approves the contract to spend their tokens.
2. User calls openDeposit(planId, amount).
3. Contract checks: plan is enabled, amount is within min/max limits.
4. Contract transfers tokens from user to itself (holds the principal).
5. Contract mints an NFT to the user with a unique depositId.

6. Deposit status is set to Active. maturityAt = block.timestamp + tenorDays * 86400.

```
The APR and penalty are snapshotted at this moment.
Future plan updates do not affect this deposit.
```

3.2 Withdraw at Maturity

User waits until the term ends, then calls withdrawAtMaturity(depositId).

The interest formula (simple interest):

```
interest = (principal * aprBpsAtOpen * tenorSeconds)
           / (365 * 24 * 3600 * 10,000)

-- tenorSeconds = tenorDays * 86,400
-- Dividing by 10,000 converts basis points to a decimal rate
```

Example: Alice deposits 1,000 USDC for 90 days at 2.5% APR (250 bps):

```
tenorSeconds = 90 * 86,400 = 7,776,000
interest = (1,000,000,000 * 250 * 7,776,000) / (31,536,000 * 10,000)
         ~ 6,164,383 units = ~6.16 USDC
Alice receives: 1,000 + 6.16 = 1,006.16 USDC
```

Important: the interest is paid from the VaultManager (not from the principal pool). Make sure the vault has enough funds before paying out.

3.3 Early Withdrawal

User withdraws before the maturity date. They receive no interest, and a penalty is deducted from the principal.

```
penalty = (principal * penaltyBpsAtOpen) / 10,000
user receives = principal - penalty
penalty goes to feeReceiver (set by admin)
```

Example: Alice withdraws early from a 1,000 USDC deposit with 5% penalty (500 bps):

```
penalty = (1,000,000,000 * 500) / 10,000 = 50,000,000 = 50 USDC
Alice receives: 1,000 - 50 = 950 USDC
feeReceiver receives: 50 USDC
Interest = 0
```

3.4 Manual Renew

When a deposit reaches maturity, the user can choose to renew instead of withdrawing.

1. User calls renewDeposit(depositId, newPlanId) on or after maturityAt.
2. Contract calculates the interest earned on the old deposit.
3. New principal = old principal + interest (interest is compounded into the principal).
4. A new deposit NFT is minted using the new plan's rate.
5. Old deposit status is set to ManualRenewed.

Example: Alice renews to a 180-day plan at 2.5% APR:

```
Old: 1,000 USDC, 90 days, ~6.16 USDC interest
New principal: 1,006.16 USDC
New tenor: 180 days
New maturityAt: now + 180 days
```

3.5 Auto Renew

If the user does nothing for 3 days after maturity (the grace period), the contract automatically renews their deposit.

Rules for auto-renew:

- Same tenor as the original deposit.
- APR is locked to the original aprBpsAtOpen - not the current plan rate. This protects the user if the admin has lowered the rate.
- New principal = old principal + interest.
- A bot (off-chain) calls autoRenewDeposit(depositId) to trigger this.

```
Grace period example: maturityAt = Day 90. Grace period ends Day 93.
If user does not act by Day 93, the bot triggers auto-renew.
The user is protected - they keep their original APR for the renewed term.
```

4. Admin Functions

The admin manages the system through VaultManager and plan configuration. The admin cannot modify existing deposits.

Function	Description
createPlan(...)	Create a new saving plan with tenor, APR, limits, and penalty.
updatePlan(planId, newAprBps)	Change the APR of a plan. Only affects new deposits.
enablePlan / disablePlan	Toggle whether users can open new deposits for a plan.
fundVault(amount)	Deposit tokens into the vault to cover interest payments.
withdrawVault(amount)	Remove tokens from the vault (within safe limits).

Function	Description
setFeeReceiver(address)	Set the address that receives early-withdrawal penalties.
pause() / unpause()	Emergency stop - prevents all withdrawals when paused.

5. Required Events

Your contracts must emit the following events. These allow frontends and indexers to track all activity.

```
PlanCreated(planId, tenorDays, aprBps)
PlanUpdated(planId, newAprBps)
DepositOpened(depositId, owner, planId, principal, maturityAt, aprBpsAtOpen)
Withdrawn(depositId, owner, principal, interest, isEarly)
Renewed(oldDepositId, newDepositId, newPrincipal, newPlanId)
```

6. Business Rules Summary

These rules must hold at all times:

1. APR and penalty are snapshotted at deposit open. Admin changes to a plan never affect existing deposits.
2. Interest uses simple interest only - no compounding within a single deposit term.
3. Early withdrawal gives zero interest. Penalty goes to feeReceiver.
4. Auto-renew preserves the original APR, protecting the user from rate decreases.
5. Interest is always paid from the vault. Base rule: if the vault has insufficient funds, the withdrawal must revert. (A better design is possible - see Creative Challenge C1 in Section 8.3.)
6. When paused, no withdrawals or renewals are allowed (emergency protection).
7. Admin cannot alter a deposit that is already open.

7. Deliverables

7.1 Smart Contracts

- **MockUSDC.sol** - ERC20 token, 6 decimals, mintable for testing.
- **VaultManager.sol** - vault funding, fee receiver, pause/unpause.
- **SavingCore.sol** - plan management, deposit logic, withdraw, renew (manual + auto), ERC721 NFT minting.

7.2 Tests

Write a full test suite using Hardhat + ethers.js or Hardhat + Waffle. Coverage must be above 90%.

Minimum test cases:

- createPlan: valid plan, disabled plan, invalid APR.

- openDeposit: happy path, below min, above max, disabled plan.
- withdrawAtMaturity: correct interest, too early, already withdrawn.
- earlyWithdraw: correct penalty, no interest paid.
- renewDeposit (manual): correct new principal, status update.
- autoRenewDeposit: before grace period (should fail), after grace period, APR locked.
- Vault: fund, withdraw, insufficient vault for interest payout.
- Pause: withdraw blocked when paused.

7.3 Frontend (Demo)

A simple React frontend that connects to MetaMask and lets a user:

- View available plans.
- Open a deposit.
- View their active deposits.
- Withdraw or renew a deposit.

7.4 Design Answers

A section in your README.md called "Design Answers" where you answer the open questions from Section 8, in your own words. Short answers are fine (3–6 sentences per question). Copying answers from another student is easy to detect, because your answers must match your own code.

8. Open Questions & Personal Variant

This section makes every submission different. There is no single correct answer - you must make a design choice, write it in your README, and defend it in the oral check. Your grade depends on how well your reasoning matches your code, not on which option you picked.

8.1 Personal Variant (based on your Student ID)

Let A = the last digit of your Student ID, and B = the second-to-last digit. Use these values in your contracts and tests:

Parameter	Your value	Example (ID ends in ...47)
Grace period (auto-renew)	(A mod 3) + 2 days	A=7 ⟶ (7 mod 3)+2 = 3 days
Default plan APR	(200 + A * 25) bps	A=7 ⟶ 375 bps = 3.75%
Early withdraw penalty	(300 + B * 50) bps	B=4 ⟶ 500 bps = 5.00%
Default plan tenor	B is even ⟶ 90 days; B is odd ⟶ 180 days	B=4 ⟶ 90 days

Write your ID digits and computed values at the top of your README. Your tests must use YOUR values, and the numbers in your demo video must match them.

8.2 Open Design Questions

Answer ALL questions below in your README. Be ready to explain any of them, live, with your own code on the screen.

1. Transferable certificate: The deposit NFT can be transferred. If Alice sells her NFT to Bob before maturity, who can withdraw - Alice or Bob? Is this behavior good or dangerous? Show the exact line in your code that decides this.
2. Empty vault: A user reaches maturity but the vault does not have enough money for the interest. The spec says "revert". What problem does this create for the user, and what alternative design could you offer (for example: pay principal only, or wait in a queue)? Which one did you choose to follow, and why?
3. Dead bot: The auto-renew bot goes offline for one month. What happens to deposits that passed the grace period? Does the user lose anything? Propose one change that protects the user in this case.
4. Rounding dust: The interest formula uses integer division, so some tiny amount is always lost to rounding. In your design, who keeps this dust - the user or the vault? Can the rounding ever cause a revert or a wrong balance? Prove your answer with one of your test cases.
5. Boundary times: At the exact second of maturityAt, is a withdrawal "early" or "at maturity"? At the exact end of the grace period, can the user still manually renew? Show the comparison operators (>= or >) you used, and explain each choice.
6. Disabled plan with active deposits: The admin disables a plan while many deposits from that plan are still active. What can those users still do? Can they still manually renew INTO the disabled plan? Justify your rule.
7. Attack thinking: Describe one realistic attack on your system (for example: reentrancy on withdraw, double withdraw, or a fake token) and show the exact mechanism in your code that stops it.

Note for grading: During the oral check, the teacher will pick 2–3 questions at random and may change the numbers ("what if the penalty was 0 bps?"). If you cannot explain your own code, the related points are lost, even if the code works.

8.3 Creative Challenges (Bonus, up to +10)

The base spec is simple on purpose, and it does NOT cover every real-world case. Here is your chance to think like a real protocol designer. Each challenge you implement AND test gives +5 bonus points (maximum +10). The final score is capped at 100, so bonus points can recover points you lost elsewhere.

ID	Challenge	The problem in the base spec
C1	Principal is always safe. Change the design so a user can ALWAYS take back their principal at maturity, even when the vault is empty (for example: pay principal now, let the user claim interest later when the vault is funded).	The base rule says "revert" when the vault cannot pay interest. This means the bank can lock the user's own money forever just by never funding the vault. That is unfair to the user.
C2	Solvency guard. Block withdrawVault so the admin cannot take out money that is already "promised" as interest to active deposits. You must track how much interest is owed.	The base spec lets the admin drain the vault at any time. Deposits that were safe yesterday can become unpayable today.

ID	Challenge	The problem in the base spec
C3	Partial early withdrawal. Let a user withdraw only part of the principal early. Penalty applies only to the withdrawn part; the rest keeps earning interest.	The base spec is all-or-nothing. A user who needs 10% of their money must break 100% of the deposit and lose the full penalty.
C4	Top-up deposit. Let a user add more principal to an active deposit. You must decide how the interest is calculated fairly for the new amount.	The base spec forces the user to open a second deposit with today's (maybe lower) APR. There is no fair way to grow an existing deposit.
C5	Your own idea. Anything that fixes a real gap you found in the spec. Describe the gap and your fix in the README first - a feature without a clear problem gets no bonus.	The spec cannot cover everything. Finding the gap yourself is the most valuable skill in this course.

Rules for bonus: (1) the base flows in Section 3 must still pass all tests; (2) each challenge needs its own tests; (3) each challenge needs a short README note: what problem, what solution, what trade-off. No tests or no note = no bonus.

9. Evaluation Criteria

Criteria	Points	Notes
Correct interest & penalty math	20	Formula accuracy, using YOUR personal variant values
APR/penalty snapshot (immutable per deposit)	15	Key invariant
Auto-renew with APR lock & grace period	15	Edge case handling
Vault management & pause/unpause	10	Admin flows
Test coverage > 90%	15	Including edge cases
Open questions + oral defense	10	Own words, matches your code
Frontend demo	10	UX & contract integration
Code quality & event emissions	5	Readability, NatSpec
Total	100	
Creative challenges (Section 8.3)	+5 each, max +10	Bonus. Final score is capped at 100.

10. Hints & Tips

These are not required approaches - they are suggestions to help you avoid common mistakes.

- Use block.timestamp for all time checks. In tests, use Hardhat's time helpers to fast-forward.
- Store all token amounts in the smallest unit (wei-equivalent). For 6-decimal USDC, 1 USDC = 1,000,000 units.
- The interest formula uses integer division. Make sure you multiply before dividing to avoid precision loss.
- The vault and principal are separate. Keep them conceptually and ideally architecturally separate.
- For auto-renew: check that now >= maturityAt + grace period before allowing it (use YOUR grace period from Section 8.1).
- Think carefully about what happens if a user tries to withdraw twice.
- Consider using OpenZeppelin's ERC721, Ownable, and Pausable - but understand what they do.

```
Precision tip: always compute (a * b * c) / d  not  (a / d) * b * c
The first keeps precision; the second can truncate to zero for small values.
```

11. Submission

- GitHub repository with all source code.
- README.md explaining how to run tests and deploy locally, plus your "Design Answers" section (see 7.4) and your personal variant values (see 8.1).
- A short demo video (3–5 minutes) walking through the frontend.